

```
In [1]: %load_ext autoreload
        %autoreload 2
```

```
In [2]: import os
import sys

current_dir = os.getcwd()

if current_dir.endswith('notebooks'):
    project_root = os.path.abspath(os.path.join(current_dir, '..'))
    os.chdir(project_root)
elif current_dir.endswith('thesis'):
    project_root = os.path.abspath(os.path.join(current_dir, 'Initial_project'))
    os.chdir(project_root)
else:
    project_root = current_dir

if project_root not in sys.path:
    sys.path.append(project_root)

print('Current working dir: ', os.getcwd())
```

Current working dir: c:\Users\Andrew K\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project

```
In [3]: from src.data.dataloader import DataModule

config_path = 'configs/config.yml'
print(f'Loading the configuration data from {config_path}')
data_module = DataModule(config_path=config_path)
data_module.setup()
print(f'Found {data_module.classes} classes')
```

Loading the configuration data from configs/config.yml
Found ['fake', 'real'] classes

```
In [4]: train_loader = data_module.get_train_dataloader()
val_loader = data_module.get_val_dataloader()
print(f'Found {len(train_loader)} train batches')
print(f'Found {len(val_loader)} validation batches')
print('-' * 40)
images, labels = next(iter(train_loader))
print(f'Shape of images batch: {images.shape}')
print(f'Shape of labels batc: {labels.shape}')
print(f'First 5 labels: {labels[:5]}')
```

Found 50 train batches
Found 13 validation batches

c:\Users\Andrew K\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\.venv\Lib\site-packages\torch\utils\data\dataloader.py:1102: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.

```
super().__init__(loader)
Shape of images batch: torch.Size([32, 3, 224, 224])
Shape of labels batc: torch.Size([32])
First 5 labels: tensor([1, 1, 0, 0, 0])
```

```
In [14]: import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from tqdm import tqdm
from src.models.vit_pretrained_classifier import PretrainedVit

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

model = PretrainedVit(num_classes=2, freeze_base=True)
model = model.to(device)
```

Using device: cpu

```
In [15]: EPOCHS = 5
LEARNING_RATE = 1e-3

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.model.heads.head.parameters(), lr=LEARNING_RATE)

history_epochs = []
history_train_loss = []
history_val_acc = []
```

```
In [18]: print('Start Training')
print('-'*40)

for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0

    training_loop = tqdm(train_loader, desc=f'Epoch {epoch+1}/{EPOCHS} [Train]')

    for images, labels in training_loop:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        training_loop.set_postfix(loss=loss.item())

    avg_train_loss = running_loss / len(train_loader)

    model.eval()
    correct_preds = 0
    total_preds = 0

    val_loop = tqdm(val_loader, desc=f'Epoch {epoch+1}/{EPOCHS} [Val]', leave=False)
    with torch.no_grad():
        for images, labels in val_loop:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
```

```

_, predicted = torch.max(outputs, 1)
total_preds += labels.size(0)
correct_preds += (predicted == labels).sum().item()

val_accuracy = correct_preds / total_preds
history_epochs.append(epoch + 1)
history_train_loss.append(avg_train_loss)
history_val_acc.append(val_accuracy)

print(f'Epoch {epoch+1} finished | Train Loss: {avg_train_loss:.4f} | Validation acc: {val_accuracy:.4f}')

```

Start Training

Epoch 1/5 [Train]: 100%|██████████| 50/50 [08:41<00:00, 10.44s/it, loss=0.369]

Epoch 1 finished | Train Loss: 0.4608 | Validation acc: 0.7500

Epoch 2/5 [Train]: 100%|██████████| 50/50 [10:05<00:00, 12.12s/it, loss=0.32]

Epoch 2 finished | Train Loss: 0.4117 | Validation acc: 0.7575

Epoch 3/5 [Train]: 100%|██████████| 50/50 [09:39<00:00, 11.59s/it, loss=0.357]

Epoch 3 finished | Train Loss: 0.3763 | Validation acc: 0.7800

Epoch 4/5 [Train]: 32%|███| 16/50 [03:07<06:37, 11.69s/it, loss=0.328]

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[18], line 15
    11     for images, labels in training_loop:
    12         images, labels = images.to(device), labels.to(device)
    13         optimizer.zero_grad()
    14
--> 15         outputs = model(images)
    16         loss = criterion(outputs, labels)
    17
    18         loss.backward()

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\venv\Lib\site-packages\torch\nn\modules\module.py:1778, in Module._wrapped_call_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\venv\Lib\site-packages\torch\nn\modules\module.py:1789, in Module._call_impl(self, *args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or _global_backward_hooks
    1788         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\src\models\vit_pretrained_classifier.py:24, in PretrainedVit.forward(self, x)
    23 def forward(self, x):
--> 24     return self.model(x)

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\venv\Lib\site-packages\torch\nn\modules\module.py:1778, in Module._wrapped_call_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\venv\Lib\site-packages\torch\nn\modules\module.py:1789, in Module._call_impl(self, *args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or _global_backward_hooks
    1788         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project

```

```

ect\.venv\Lib\site-packages\torchvision\models\vision_transformer.py:298, in Vi
onTransformer.forward(self, x)
    295 batch_class_token = self.class_token.expand(n, -1, -1)
    296 x = torch.cat([batch_class_token, x], dim=1)
--> 298 x = self.encoder(x)
    300 # Classifier "token" as used by standard language architectures
    301 x = x[:, 0]

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_proj
ect\.venv\Lib\site-packages\torch\nn\modules\module.py:1778, in Module._wrapped_c
all_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type: ignore[mis
c]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_proj
ect\.venv\Lib\site-packages\torch\nn\modules\module.py:1789, in Module._call_impl
(self, *args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or self._forward
_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or _global_backward_hooks
    1788         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_proj
ect\.venv\Lib\site-packages\torchvision\models\vision_transformer.py:157, in Enco
der.forward(self, input)
    155 torch._assert(input.dim() == 3, f"Expected (batch_size, seq_length, hidde
n_dim) got {input.shape}")
    156 input = input + self.pos_embedding
--> 157 return self.ln(self.layers(self.dropout(input)))

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_proj
ect\.venv\Lib\site-packages\torch\nn\modules\module.py:1778, in Module._wrapped_c
all_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type: ignore[mis
c]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_proj
ect\.venv\Lib\site-packages\torch\nn\modules\module.py:1789, in Module._call_impl
(self, *args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or self._forward
_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or _global_backward_hooks
    1788         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_proj
ect\.venv\Lib\site-packages\torch\nn\modules\container.py:253, in Sequential.forw

```

```

ard(self, input)
    249 """
    250 Runs the forward pass.
    251 """
    252 for module in self:
--> 253     input = module(input)
    254 return input

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\.venv\Lib\site-packages\torch\nn\modules\module.py:1778, in Module._wrapped_call_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\.venv\Lib\site-packages\torch\nn\modules\module.py:1789, in Module._call_impl(self, *args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or _global_backward_hooks
    1788         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\.venv\Lib\site-packages\torchvision\models\vision_transformer.py:118, in EncoderBlock.forward(self, input)
    115 x = x + input
    117 y = self.ln_2(x)
--> 118 y = self.mlp(y)
    119 return x + y

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\.venv\Lib\site-packages\torch\nn\modules\module.py:1778, in Module._wrapped_call_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\.venv\Lib\site-packages\torch\nn\modules\module.py:1789, in Module._call_impl(self, *args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or _global_backward_hooks
    1788         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

```

```

File ~\OneDrive - University of Patras\Επιφάνεια εργασίας\UNI\thesis\Initial_project\.venv\Lib\site-packages\torch\nn\modules\container.py:253, in Sequential.forward

```

```
ard(self, input)
249 """
250 Runs the forward pass.
251 """
252 for module in self:
--> 253     input = module(input)
254 return input
```

KeyboardInterrupt: